

Day 3 – Document Extractor using Structured Outputs + Function Calling Patterns

Goal: Extract structured, validated data from B2B invoices using the Gemini API's `response_schema` feature and Pydantic models. By the end of this project you will have a working extraction pipeline, 10 synthetic invoice samples, ground truth data, evaluation metrics, and a written summary of what worked and what didn't.

Table of Contents

1. [Project Overview](#)
 2. [Folder Structure Setup](#)
 3. [Schema Design](#)
 4. [Complete Code](#)
 5. [Sample Documents](#)
 6. [Ground Truth](#)
 7. [DeepSeek Prompts](#)
 8. [Evaluation Logic](#)
 9. [Accuracy Summary Template](#)
 10. [README](#)
 11. [learnings.md Template](#)
 12. [Troubleshooting](#)
-

1. Project Overview

This project simulates a real workflow at Wohlig: a client uploads a scanned or digital invoice, and your pipeline must extract structured fields from it — vendor details, buyer details, line items, totals, status — and return a validated JSON object.

Why Gemini?

Gemini's `response_schema` parameter forces the model to return JSON that conforms exactly to a schema you define. This eliminates the need for regex parsing or post-processing hacks. Combined with Pydantic, you get type validation for free.

Why B2B Invoice?

It naturally covers every schema complexity you need to practice:

- Nested objects (vendor address, buyer address)
 - Lists (line items)
 - Enums (invoice status)
 - Optional fields (purchase order number, due date, notes)
 - Numeric precision (totals)
-

2. Folder Structure Setup

Final structure

```
/extractor/  
  main.py  
  schema.py  
  evaluate.py  
  generate_samples.py  
  requirements.txt  
  README.md  
  ground_truth.jsonl  
  field_accuracy.csv      ← generated by evaluate.py  
  accuracy_summary.md    ← generated/filled manually  
  learnings.md  
  samples/  
    invoice_001.txt  
    invoice_002.txt  
    invoice_003.txt  
    invoice_004.txt  
    invoice_005.txt  
    invoice_006.txt  
    invoice_007.txt  
    invoice_008.txt  
    invoice_009.txt  
    invoice_010.txt  
  outputs/  
    invoice_001.extracted.json  
    ...
```

Terminal commands

Run these from whatever parent directory you want. Every command is copy-paste ready.

```
mkdir extractor  
cd extractor  
  
mkdir samples outputs  
  
touch main.py schema.py evaluate.py generate_samples.py  
touch requirements.txt README.md ground_truth.jsonl  
touch accuracy_summary.md learnings.md  
  
touch samples/invoice_001.txt samples/invoice_002.txt  
samples/invoice_003.txt \  
  samples/invoice_004.txt samples/invoice_005.txt  
samples/invoice_006.txt \  
  samples/invoice_007.txt samples/invoice_008.txt  
samples/invoice_009.txt \  
  samples/invoice_010.txt
```

3. Schema Design

Concept

`response_schema` tells Gemini: "Your output must be a JSON object with exactly these fields, in exactly these types." Pydantic then validates the parsed JSON against the same schema on the Python side. If either step fails, you reject the output.

Field-by-field explanation

Field	Type	Notes
<code>invoice_number</code>	<code>str</code>	Unique identifier on the invoice
<code>invoice_date</code>	<code>str</code>	ISO date string (YYYY-MM-DD)
<code>due_date</code>	<code>Optional[str]</code>	May be absent; always optional
<code>status</code>	<code>InvoiceStatus</code> enum	One of five fixed values
<code>vendor</code>	<code>Party</code>	Nested object — seller
<code>buyer</code>	<code>Party</code>	Nested object — purchaser
<code>line_items</code>	<code>List[LineItem]</code>	One entry per product/service
<code>subtotal</code>	<code>float</code>	Before tax
<code>tax_amount</code>	<code>float</code>	Tax component
<code>total_amount</code>	<code>float</code>	Final amount due
<code>currency</code>	<code>str</code>	Three-letter ISO code (USD, EUR, INR...)
<code>purchase_order_number</code>	<code>Optional[str]</code>	Buyer's PO ref, often absent
<code>notes</code>	<code>Optional[str]</code>	Any footer notes or payment instructions

Party object (used for both vendor and buyer):

Field	Type	Notes
<code>name</code>	<code>str</code>	Company name
<code>address</code>	<code>Address</code>	Nested address
<code>tax_id</code>	<code>Optional[str]</code>	GST, VAT, EIN, etc.
<code>email</code>	<code>Optional[str]</code>	Contact email

Address object:

Field	Type
<code>street</code>	<code>str</code>
<code>city</code>	<code>str</code>

Field	Type
state	Optional[str]
country	str
postal_code	Optional[str]

LineItem object:

Field	Type	Notes
description	str	What was sold
quantity	float	Units
unit_price	float	Price per unit
total	float	quantity × unit_price

InvoiceStatus enum:

```
paid | unpaid | overdue | partially_paid | unknown
```

4. Complete Code

schema.py

```
# schema.py
from __future__ import annotations
from enum import Enum
from typing import List, Optional
from pydantic import BaseModel, Field

class InvoiceStatus(str, Enum):
    paid = "paid"
    unpaid = "unpaid"
    overdue = "overdue"
    partially_paid = "partially_paid"
    unknown = "unknown"

class Address(BaseModel):
    street: str
    city: str
    state: Optional[str] = None
    country: str
    postal_code: Optional[str] = None
```

```

class Party(BaseModel):
    name: str
    address: Address
    tax_id: Optional[str] = None
    email: Optional[str] = None

class LineItem(BaseModel):
    description: str
    quantity: float
    unit_price: float
    total: float

class Invoice(BaseModel):
    invoice_number: str
    invoice_date: str = Field(description="ISO date string YYYY-MM-DD")
    due_date: Optional[str] = Field(default=None, description="ISO date string YYYY-MM-DD")
    status: InvoiceStatus
    vendor: Party
    buyer: Party
    line_items: List[LineItem]
    subtotal: float
    tax_amount: float
    total_amount: float
    currency: str = Field(description="Three-letter ISO currency code, e.g. USD")
    purchase_order_number: Optional[str] = None
    notes: Optional[str] = None

```

main.py

```

# main.py
"""
Usage:
    python main.py samples/invoice_001.txt

Reads a plain-text invoice, sends it to Gemini with a strict
response_schema,
validates the returned JSON with Pydantic, prints it, and saves it to
outputs/.
"""

import json
import os
import sys
from pathlib import Path

```

```

import google.generativeai as genai
from pydantic import ValidationError

from schema import Invoice

# — Configuration


---



GEMINI_MODEL = "gemini-1.5-flash"
OUTPUT_DIR = Path("outputs")

def get_api_key() -> str:
    key = os.environ.get("GEMINI_API_KEY", "").strip()
    if not key:
        print(
            "\n[ERROR] GEMINI_API_KEY environment variable is not set.\n"
            "Fix: export GEMINI_API_KEY='your-key-here'\n"
            "Get a key at: https://aistudio.google.com/app/apikey\n"
        )
        sys.exit(1)
    return key

# — Schema conversion


---



def pydantic_to_gemini_schema(model) -> dict:
    """
    Convert a Pydantic model's JSON schema into the format Gemini expects
    for response_schema. Gemini uses a restricted subset of JSON Schema.
    """
    raw = model.model_json_schema()
    return _clean_schema(raw, raw.get("$defs", {}))

def _clean_schema(schema: dict, defs: dict) -> dict:
    """
    Recursively resolve $ref references and strip keys Gemini doesn't
    accept.
    """
    # Resolve $ref
    if "$ref" in schema:
        ref_name = schema["$ref"].split("/")[-1]
        resolved = defs.get(ref_name, {})
        return _clean_schema(resolved, defs)

    result = {}

    if "type" in schema:
        result["type"] = schema["type"].upper() # Gemini wants uppercase
strings

```

```

if "properties" in schema:
    result["type"] = "OBJECT"
    result["properties"] = {
        k: _clean_schema(v, defs)
        for k, v in schema["properties"].items()
    }
    # Mark required fields
    if "required" in schema:
        result["required"] = schema["required"]

if "items" in schema:
    result["type"] = "ARRAY"
    result["items"] = _clean_schema(schema["items"], defs)

if "enum" in schema:
    result["type"] = "STRING"
    result["enum"] = schema["enum"]

# anyOf with null means Optional – use the non-null type
if "anyOf" in schema:
    non_null = [s for s in schema["anyOf"] if s.get("type") != "null"]
    if non_null:
        return _clean_schema(non_null[0], defs)
    result["type"] = "STRING" # fallback

if schema.get("type") in ("number", "float"):
    result["type"] = "NUMBER"

if schema.get("type") == "integer":
    result["type"] = "INTEGER"

if schema.get("type") == "string":
    result["type"] = "STRING"

if schema.get("type") == "boolean":
    result["type"] = "BOOLEAN"

if schema.get("type") == "array":
    result["type"] = "ARRAY"

return result

```

— Extraction

EXTRACTION_PROMPT = """\nYou are a document extraction assistant. Extract all invoice fields from the document below and return them as structured JSON matching the schema exactly.

Rules:

- invoice_date and due_date must be ISO format YYYY-MM-DD if present.

- currency must be a three-letter ISO code (e.g. USD, EUR, GBP, INR).
- status must be one of: paid, unpaid, overdue, partially_paid, unknown.
- subtotal, tax_amount, and total_amount must be numeric (float).
- line_item totals must equal quantity × unit_price (do not round arbitrarily).
- If a field is not present in the document and is optional, omit it or use null.
- Do not invent data. If you cannot determine a value, use null for optional fields or your best reasonable extraction for required fields.

DOCUMENT:

```
{document_text}
"""
```

```
def extract_invoice(file_path: Path) -> Invoice:
    if not file_path.exists():
        print(f"\n[ERROR] File not found: {file_path}\n")
        sys.exit(1)

    document_text = file_path.read_text(encoding="utf-8")
    if not document_text.strip():
        print(f"\n[ERROR] File is empty: {file_path}\n")
        sys.exit(1)

    api_key = get_api_key()
    genai.configure(api_key=api_key)

    # Build Gemini response schema from Pydantic model
    response_schema = pydantic_to_gemini_schema(Invoice)

    model = genai.GenerativeModel(
        model_name=GEMINI_MODEL,
        generation_config=genai.GenerationConfig(
            response_mime_type="application/json",
            response_schema=response_schema,
        ),
    )

    prompt = EXTRACTION_PROMPT.format(document_text=document_text)

    try:
        response = model.generate_content(prompt)
    except Exception as e:
        print(f"\n[ERROR] Gemini API call failed: {e}\n")
        sys.exit(1)

    raw_text = response.text
    if not raw_text or not raw_text.strip():
        print("\n[ERROR] Gemini returned an empty response.\n")
        sys.exit(1)

    # Parse JSON
```



```

try:
    raw_data = json.loads(raw_text)
except json.JSONDecodeError as e:
    print(f"\n[ERROR] Gemini returned malformed JSON.\nError: {e}\nRaw
response:\n{raw_text}\n")
    sys.exit(1)

# Validate with Pydantic
try:
    invoice = Invoice.model_validate(raw_data)
except ValidationError as e:
    print(f"\n[ERROR] Pydantic validation failed.\nValidation
errors:\n{e}\n")
    print(f"Raw data received:\n{json.dumps(raw_data, indent=2)}")
    sys.exit(1)

return invoice

```

— Save output

```

def save_output(invoice: Invoice, input_path: Path) -> Path:
    OUTPUT_DIR.mkdir(exist_ok=True)
    stem = input_path.stem # e.g. "invoice_001"
    out_path = OUTPUT_DIR / f"{stem}.extracted.json"
    out_path.write_text(
        invoice.model_dump_json(indent=2, exclude_none=False),
        encoding="utf-8",
    )
    return out_path

```

— Entrypoint

```

def main():
    if len(sys.argv) != 2:
        print("Usage: python main.py samples/invoice_001.txt")
        sys.exit(1)

    input_path = Path(sys.argv[1])
    print(f"\nExtracting: {input_path}")

    invoice = extract_invoice(input_path)
    out_path = save_output(invoice, input_path)

    print(f"\n— Extracted Invoice —————")
    print(invoice.model_dump_json(indent=2))
    print(f"\n— Saved to: {out_path}\n")

```

```
if __name__ == "__main__":
    main()
```

evaluate.py

```
# evaluate.py
"""
Evaluates extraction accuracy by comparing extracted JSON files against
ground truth.

Usage:
    python evaluate.py

Reads:
    ground_truth.jsonl
    outputs/invoice_XXX.extracted.json (one per sample)

Writes:
    field_accuracy.csv
    (prints accuracy summary to stdout)
"""

import csv
import json
from pathlib import Path

GROUND_TRUTH_FILE = Path("ground_truth.jsonl")
OUTPUTS_DIR = Path("outputs")
CSV_OUTPUT = Path("field_accuracy.csv")

# — Flattening

def flatten(obj, prefix=""):
    """
    Recursively flatten a nested dict/list into dot-notation keys.

    Examples:
        {"vendor": {"name": "Acme"}} → {"vendor.name": "Acme"}
        {"line_items": [{"description": "X"}]} → {"line_items": <json>}

    Lists are stored as their full JSON string for comparison.
    """
    items = {}
    if isinstance(obj, dict):
        for k, v in obj.items():
            full_key = f"{prefix}.{k}" if prefix else k
            if isinstance(v, (dict, list)):
                items[full_key] = flatten(v, full_key)
            else:
                items[full_key] = v
    elif isinstance(obj, list):
        items[" "] = json.dumps(obj)
    else:
        items[" "] = obj
    return items
```

```

        items.update(flatten(v, full_key))
    else:
        items[full_key] = v
elif isinstance(obj, list):
    # Store list fields as canonical JSON strings for exact comparison
    items[prefix] = json.dumps(obj, sort_keys=True)
else:
    items[prefix] = obj
return items

```

— Comparison

```

def compare(extracted: dict, ground_truth: dict) -> list[dict]:
    """
    Compare extracted and ground truth field by field.
    Returns a list of result rows for the CSV.
    """
    flat_extracted = flatten(extracted)
    flat_truth = flatten(ground_truth)

    all_keys = set(flat_extracted.keys()) | set(flat_truth.keys())
    rows = []

    for field in sorted(all_keys):
        ext_val = flat_extracted.get(field, None)
        gt_val = flat_truth.get(field, None)

        # Normalize for comparison: strip whitespace, lowercase strings
        def norm(v):
            if v is None:
                return None
            if isinstance(v, str):
                return v.strip().lower()
            if isinstance(v, float):
                return round(v, 2)
            return v

        exact_match = norm(ext_val) == norm(gt_val)
        rows.append({
            "field_name": field,
            "extracted_value": ext_val,
            "ground_truth_value": gt_val,
            "exact_match": exact_match,
        })

    return rows

```

— Main

```

def load_ground_truth() -> list[dict]:

```

```

if not GROUND_TRUTH_FILE.exists():
    print(f"[ERROR] {GROUND_TRUTH_FILE} not found.")
    return []
records = []
with GROUND_TRUTH_FILE.open(encoding="utf-8") as f:
    for line in f:
        line = line.strip()
        if line:
            records.append(json.loads(line))
return records

def main():
    ground_truths = load_ground_truth()
    if not ground_truths:
        print("No ground truth records found. Exiting.")
        return

    all_rows = []
    missing_files = []

    for gt_record in ground_truths:
        sample_id = gt_record["sample_id"]
        file_name = gt_record["file_name"] # e.g. "invoice_001.txt"
        stem = Path(file_name).stem # "invoice_001"
        extracted_path = OUTPUTS_DIR / f"{stem}.extracted.json"

        if not extracted_path.exists():
            missing_files.append(str(extracted_path))
            continue

        with extracted_path.open(encoding="utf-8") as f:
            extracted = json.load(f)

        expected = gt_record["expected"]
        rows = compare(extracted, expected)

        for row in rows:
            row["sample_id"] = sample_id
            all_rows.append(row)

    if missing_files:
        print(f"\n[WARNING] Missing extracted files (run main.py first):")
        for mf in missing_files:
            print(f"    {mf}")

    if not all_rows:
        print("No comparison data. Nothing to write.")
        return

    # Write CSV
    fieldnames = ["sample_id", "field_name", "extracted_value",
                  "ground_truth_value", "exact_match"]
    with CSV_OUTPUT.open("w", newline="", encoding="utf-8") as f:

```

```

writer = csv.DictWriter(f, fieldnames=fieldnames)
writer.writeheader()
writer.writerows(all_rows)

print(f"\nWrote {len(all_rows)} rows to {CSV_OUTPUT}")

# Print summary
total = len(all_rows)
correct = sum(1 for r in all_rows if r["exact_match"])
overall = correct / total * 100 if total > 0 else 0
print(f"\nOverall field accuracy: {correct}/{total} = {overall:.1f}%")

# Per-field accuracy
field_stats: dict[str, dict] = {}
for row in all_rows:
    fn = row["field_name"]
    if fn not in field_stats:
        field_stats[fn] = {"correct": 0, "total": 0}
    field_stats[fn]["total"] += 1
    if row["exact_match"]:
        field_stats[fn]["correct"] += 1

print("\nPer-field accuracy:")
print(f"{'Field':<40} {'Correct':>8} {'Total':>7} {'Acc%':>8}")
print("-" * 65)
for fn, stats in sorted(field_stats.items()):
    acc = stats["correct"] / stats["total"] * 100
    print(f"{'fn':<40} {'stats['correct']':>8} {'stats['total']':>7}
{'acc:>7.1f}%"")

if __name__ == "__main__":
    main()

```

generate_samples.py

```

# generate_samples.py
"""
Uses Gemini to generate 10 synthetic B2B invoice text files and saves them
into samples/. Run this once if you want Gemini-generated variation
instead of the hand-written samples in the guide.

Usage:
    python generate_samples.py
"""

import os
import sys
from pathlib import Path
import google.generativeai as genai

```

```
SAMPLES_DIR = Path("samples")
GEMINI_MODEL = "gemini-1.5-flash"
```

```
GENERATION_PROMPT = """\
Generate a realistic plain-text B2B invoice as it would appear in a scanned
or
digital document. The invoice should be for {description}.
```

Requirements:

- Invoice number in format INV-XXXX
- Invoice date and due date in a natural format (e.g. "15 March 2024")
- Vendor name, address, tax ID, email
- Buyer name, address
- {extra_requirements}
- Line items with description, quantity, unit price, and line total
- Subtotal, tax amount (label it clearly), and total amount
- Currency: {currency}
- Status note at the bottom: {status}
- Realistic company names, addresses, and product/service names

```
Return only the plain text of the invoice. No JSON. No markdown. No
explanation.
"""
```

```
SAMPLE_SPECS = [
    {"id": "001", "description": "software consulting services",
"currency": "USD", "status": "UNPAID",
    "extra": "Include a purchase order number PO-2024-0091"},
    {"id": "002", "description": "office furniture and supplies",
"currency": "EUR", "status": "PAID",
    "extra": "Add a note that payment was received"},
    {"id": "003", "description": "cloud hosting services with multiple line
items (at least 6)", "currency": "GBP", "status": "OVERDUE",
    "extra": "No purchase order number"},
    {"id": "004", "description": "marketing and design services",
"currency": "INR", "status": "PARTIALLY PAID",
    "extra": "Include a note about the remaining balance due"},
    {"id": "005", "description": "laboratory equipment", "currency": "USD",
"status": "UNPAID",
    "extra": "Omit the due date entirely"},
    {"id": "006", "description": "logistics and freight services",
"currency": "AED", "status": "PAID",
    "extra": "Include a purchase order number"},
    {"id": "007", "description": "IT hardware procurement", "currency":
"SGD", "status": "UNPAID",
    "extra": "Use unusual spacing and alignment (simulate a poorly
formatted scan)"},
    {"id": "008", "description": "legal consulting services", "currency":
"USD", "status": "OVERDUE",
    "extra": "No purchase order number. Add a late payment notice"},
    {"id": "009", "description": "advertising agency retainer", "currency":
"CAD", "status": "UNPAID",
```

```

        "extra": "Include purchase order number PO-CA-7721"},
        {"id": "010", "description": "construction materials", "currency":
"ZAR", "status": "PARTIALLY PAID",
        "extra": "Include 8 line items and a note about delivery charges"},
    ]

def get_api_key() -> str:
    key = os.environ.get("GEMINI_API_KEY", "").strip()
    if not key:
        print("[ERROR] GEMINI_API_KEY not set.")
        sys.exit(1)
    return key

def main():
    SAMPLES_DIR.mkdir(exist_ok=True)
    genai.configure(api_key=get_api_key())
    model = genai.GenerativeModel(GEMINI_MODEL)

    for spec in SAMPLE_SPECS:
        out_path = SAMPLES_DIR / f"invoice_{spec['id']}.txt"
        if out_path.exists():
            print(f"[SKIP] {out_path} already exists.")
            continue

        prompt = GENERATION_PROMPT.format(
            description=spec["description"],
            currency=spec["currency"],
            status=spec["status"],
            extra_requirements=spec["extra"],
        )
        print(f"Generating invoice_{spec['id']}.txt ...")
        try:
            response = model.generate_content(prompt)
            out_path.write_text(response.text.strip(), encoding="utf-8")
            print(f"    Saved → {out_path}")
        except Exception as e:
            print(f"    [ERROR] {e}")

if __name__ == "__main__":
    main()

```

requirements.txt

```

google-generativeai>=0.7.0
pydantic>=2.0.0

```

Install with:

```
pip install -r requirements.txt
```

5. Sample Documents

Copy each block below into its corresponding `.txt` file inside `samples/`.

`samples/invoice_001.txt`

INVOICE

Vendor:

TechPulse Solutions Inc.
1402 Innovation Drive, Suite 300
Austin, TX 78701
United States
Tax ID: 82-4719203
Email: billing@techpulse.io

Bill To:

Nexora Digital Ltd.
88 Commerce Street, Floor 4
New York, NY 10004
United States

Invoice Number: INV-2024-0091

Invoice Date: 10 January 2024

Due Date: 10 February 2024

Purchase Order: PO-2024-0091

Status: UNPAID

DESCRIPTION	QTY	UNIT PRICE	TOTAL
Senior Backend Development	80 hrs	\$150.00	\$12,000.00
API Integration Consulting	20 hrs	\$175.00	\$3,500.00
Code Review and QA Services	10 hrs	\$120.00	\$1,200.00

Subtotal:

\$16,700.00

Tax (8%):

\$1,336.00

TOTAL DUE:

\$18,036.00

Currency: USD

Payment due within 30 days. Wire transfer details available on request.

samples/invoice_002.txt

INVOICE

From:

Brunner & Meyer GmbH
Kaiserstrasse 47
60329 Frankfurt
Germany
VAT: DE298471023
Email: accounts@brunnermeyer.de

To:

Orbis Workspace S.A.
Rue de la Loi 200
1000 Brussels
Belgium

Invoice No.: INV-2024-0412

Date: 22 February 2024

Due Date: 22 March 2024

Status: PAID

Items:

Ergonomic Office Chairs (x10)	1	€3,200.00	€3,200.00
Standing Desk Frames (x5)	1	€2,750.00	€2,750.00
Desk Accessories Bundle	3	€180.00	€540.00

Subtotal:

VAT (19%):

Total:

€6,490.00

€1,233.10

€7,723.10

Currency: EUR

Payment received on 25 February 2024. Thank you for your business.

samples/invoice_003.txt

INVOICE

Vendor:

CloudNine Infrastructure Ltd.
45 Server Farm Road
London, EC2V 8RF
United Kingdom
VAT No: GB 312 9847 01
contact@cloudnine.co.uk

Client:

DataVault Analytics plc 99 Canary Wharf London, E14 5AB United Kingdom			
Invoice #: INV-2024-CN-0038 Invoice Date: 01 March 2024 Due Date: 31 March 2024 Status: OVERDUE			
Services Rendered:			
Compute Instances (c2.xlarge x4, 720 hrs)	720	£0.68	£489.60
Storage (20TB SSD, monthly)	1	£840.00	£840.00
CDN Bandwidth (8TB transfer)	8	£32.00	£256.00
Load Balancer (2 units, monthly)	2	£95.00	£190.00
Managed Database (PostgreSQL, monthly)	1	£310.00	£310.00
DDoS Protection (standard tier)	1	£120.00	£120.00
Subtotal: £2,205.60			
Tax (20%): £441.12			
TOTAL: £2,646.72			
Currency: GBP			
OVERDUE NOTICE: This invoice is past its due date. Please remit payment immediately. Late payment fee of 2% per month applies after 30 days.			

samples/invoice_004.txt

INVOICE	
Vendor: CreativeEdge Marketing Pvt. Ltd. 501 Maker Chambers IV Nariman Point, Mumbai 400021 India GSTIN: 27AAGCC4739R1Z3 Email: finance@creativeedge.in	
Bill To: BrandAxis Retail Pvt. Ltd. 14th Floor, DLF Cyber City Gurugram, Haryana 122002 India	
Invoice Number: INV-2024-CE-0189 Invoice Date: 05 March 2024 Due Date: 05 April 2024 Status: PARTIALLY PAID	

Description	Qty	Rate	Amount
Brand Strategy Consulting	1	₹75,000.00	₹75,000.00
Social Media Campaign Design	1	₹45,000.00	₹45,000.00
Video Production (30-sec ad)	2	₹60,000.00	₹120,000.00
Subtotal:		₹2,40,000.00	
GST (18%):		₹43,200.00	
Total:		₹2,83,200.00	
Currency: INR			
Note: A partial payment of ₹1,00,000.00 was received on 15 March 2024. Remaining balance of ₹1,83,200.00 is due by 05 April 2024.			

[samples/invoice_005.txt](#)

INVOICE

Seller:
PrecisionLab Instruments Corp.
2200 Research Parkway
San Diego, CA 92121
United States
EIN: 47-2381049
Email: orders@precisionlab.com

Buyer:
GenomeTech Research Institute
500 Science Boulevard
Boston, MA 02115
United States

Invoice Number: INV-2024-PL-0057
Invoice Date: 14 March 2024

Status: UNPAID

Item Description	Qty	Unit Price	Total
Centrifuge Model CF-5000	2	\$4,800.00	\$9,600.00
Micropipette Set (1000ul)	10	\$185.00	\$1,850.00
Lab Refrigeration Unit (4°C)	1	\$3,200.00	\$3,200.00

Subtotal: \$14,650.00
Tax (7.5%): \$1,098.75
Total: \$15,748.75

Currency: USD

No purchase order on file. Net 45 payment terms apply.

[samples/invoice_006.txt](#)

INVOICE

From:
Gulf Freight Express LLC
Warehouse 14, Jebel Ali Free Zone
Dubai, UAE
TRN: 100482931400003
billing@gulffreight.ae

To:
Meridian Trading Company FZCO
Office 2201, DWTC
Dubai, UAE

Invoice No.: INV-2024-GF-0302
Invoice Date: 18 March 2024
Due Date: 17 April 2024
Purchase Order: PO-MTC-20240318
Status: PAID

Service Description	Qty	Rate (AED)	Total (AED)
Air Freight – DXB to SIN	1	12,500.00	12,500.00
Customs Clearance Fees	1	1,800.00	1,800.00
Warehouse Storage (7 days)	7	350.00	2,450.00
Documentation Handling	1	600.00	600.00

Subtotal: 17,350.00 AED
VAT (5%): 867.50 AED
Total: 18,217.50 AED

Currency: AED

Payment received via bank transfer on 20 March 2024. Thank you.

samples/invoice_007.txt

I N V O I C E

Vendor :
ByteForge Hardware Pte. Ltd.
18 Tanjong Pagar Road, #05-01
Singapore 088 443
GST Reg: M9-0234817-K
sales @ byteforge.sg

Bill To :
Synergy Tech Corp.
77 Shenton Way, Level 12
Singapore 0789 03

Invoice Number : INV – 2024 – BF – 0061
Invoice Date : 22 March 2024
Due Date : 21 April 2024

Status : UNPAID

ITEMS :			
Enterprise Server (Dell R750xa)	2	SGD 8,400.00	SGD 16,800.00
Network Switch (48-port)	4	SGD 850.00	SGD 3,400.00
UPS Battery Backup Unit	2	SGD 620.00	SGD 1,240.00

Sub – total : SGD 21,440.00
GST (9%) : SGD 1,929.60
T O T A L : SGD 23,369.60

Currency : SGD

samples/invoice_008.txt

INVOICE

From:
Harrington & Cole Legal Associates LLP
850 Third Avenue, 22nd Floor
New York, NY 10022
United States
EIN: 13-5527894
Email: accounts@harringtoncole.com

To:
Vantage Capital Group Inc.
200 Park Avenue
New York, NY 10166
United States

Invoice Number: INV-2024-HC-0088
Invoice Date: 01 February 2024
Due Date: 01 March 2024
Status: OVERDUE

Description	Hours	Rate	Amount
M&A Due Diligence Review	40	\$450.00	\$18,000.00
Contract Negotiation Support	15	\$450.00	\$6,750.00
Regulatory Compliance Advisory	10	\$450.00	\$4,500.00

Subtotal: \$29,250.00
Tax (0%): \$0.00
Total: \$29,250.00

Currency: USD

OVERDUE NOTICE: This invoice was due on 01 March 2024. As of today this invoice remains unpaid. A late payment penalty of 1.5% per month is now accruing. Please contact accounts@harringtoncole.com immediately to resolve.

samples/invoice_009.txt

INVOICE

Vendor:
Pinnacle Media Group Inc.

340 King Street West, Suite 500
Toronto, ON M5V 1J2
Canada
HST: 82471-9203-RT0001
billing@pinnaclemedia.ca

Client:
Northern Lights Retail Corp.
1000 Portage Avenue
Winnipeg, MB R3G 0S8
Canada

Invoice No.: INV-2024-PM-0214
Invoice Date: 08 April 2024
Due Date: 08 May 2024
Purchase Order: PO-CA-7721
Status: UNPAID

Services:			
Advertising Retainer (April 2024)	1	CAD 12,000.00	CAD 12,000.00
Performance Analytics Report	1	CAD 2,500.00	CAD 2,500.00
Creative Asset Production	1	CAD 4,200.00	CAD 4,200.00

Subtotal: CAD 18,700.00
HST (13%): CAD 2,431.00
Total Due: CAD 21,131.00

Currency: CAD

Payment terms: Net 30. E-transfer or wire accepted.

samples/invoice_010.txt

INVOICE

Vendor:
Cape Build Materials (Pty) Ltd.
12 Industria Road
Cape Town, 7490
South Africa
VAT No.: 4320198754
Email: accounts@capebuild.co.za

Customer:
Sunrise Property Developers (Pty) Ltd.
88 Bree Street
Johannesburg, 2001
South Africa

Invoice Number: INV-2024-CB-0503
Invoice Date: 15 April 2024

Due Date: 15 May 2024
Status: PARTIALLY PAID

Description	Qty	Unit Price	Total
Concrete Blocks (Standard)	5000	ZAR 8.50	ZAR 42,500.00
River Sand (per ton)	40	ZAR 480.00	ZAR 19,200.00
Building Cement (50kg bags)	200	ZAR 145.00	ZAR 29,000.00
Steel Reinforcement Bars (12mm)	300	ZAR 85.00	ZAR 25,500.00
Waterproofing Membrane (rolls)	50	ZAR 320.00	ZAR 16,000.00
Roof Tiles (per sqm)	120	ZAR 265.00	ZAR 31,800.00
PVC Piping (4m lengths, 50mm)	100	ZAR 75.00	ZAR 7,500.00
Delivery Charges	1	ZAR 4,500.00	ZAR 4,500.00

Subtotal: ZAR 176,000.00
VAT (15%): ZAR 26,400.00
Total: ZAR 202,400.00

Currency: ZAR

Note: A partial payment of ZAR 100,000.00 was received on 20 April 2024.
Outstanding balance: ZAR 102,400.00. Please remit by 15 May 2024.

6. Ground Truth

Create `ground_truth.jsonl`. Each line is one JSON object. The file has exactly 10 lines.

```
{"sample_id": "001", "file_name": "invoice_001.txt", "expected": {"invoice_number": "INV-2024-0091", "invoice_date": "2024-01-10", "due_date": "2024-02-10", "status": "unpaid", "vendor": {"name": "TechPulse Solutions Inc.", "address": {"street": "1402 Innovation Drive, Suite 300", "city": "Austin", "state": "TX", "country": "United States", "postal_code": "78701"}, "tax_id": "82-4719203", "email": "billing@techpulse.io"}, "buyer": {"name": "Nexora Digital Ltd.", "address": {"street": "88 Commerce Street, Floor 4", "city": "New York", "state": "NY", "country": "United States", "postal_code": "10004"}, "tax_id": null, "email": null}, "line_items": [{"description": "Senior Backend Development", "quantity": 80.0, "unit_price": 150.0, "total": 12000.0}, {"description": "API Integration Consulting", "quantity": 20.0, "unit_price": 175.0, "total": 3500.0}, {"description": "Code Review and QA Services", "quantity": 10.0, "unit_price": 120.0, "total": 1200.0}], "subtotal": 16700.0, "tax_amount": 1336.0, "total_amount": 18036.0, "currency": "USD", "purchase_order_number": "PO-2024-0091", "notes": "Payment due within 30 days. Wire transfer details available on request."}} {"sample_id": "002", "file_name": "invoice_002.txt", "expected": {"invoice_number": "INV-2024-0412", "invoice_date": "2024-02-22", "due_date": "2024-03-22", "status": "paid", "vendor": {"name": "Brunner & Meyer GmbH", "address": {"street": "Kaiserstrasse 47", "city": "Frankfurt", "state": null, "country": "Germany", "postal_code": "60329"}, "tax_id": "DE298471023", "email": "accounts@brunnermeyer.de"}, "buyer": {"name": "Orbis Workspace S.A.", "address": {"street": "Rue de la Loi 200", "city":
```



```

"Brussels", "state": null, "country": "Belgium", "postal_code": "1000"},
"tax_id": null, "email": null}, {"line_items": [{"description": "Ergonomic
Office Chairs (x10)", "quantity": 1.0, "unit_price": 3200.0, "total":
3200.0}, {"description": "Standing Desk Frames (x5)", "quantity": 1.0,
"unit_price": 2750.0, "total": 2750.0}, {"description": "Desk Accessories
Bundle", "quantity": 3.0, "unit_price": 180.0, "total": 540.0}],
"subtotal": 6490.0, "tax_amount": 1233.10, "total_amount": 7723.10,
"currency": "EUR", "purchase_order_number": null, "notes": "Payment
received on 25 February 2024. Thank you for your business."}}
{"sample_id": "003", "file_name": "invoice_003.txt", "expected":
{"invoice_number": "INV-2024-CN-0038", "invoice_date": "2024-03-01",
"due_date": "2024-03-31", "status": "overdue", "vendor": {"name":
"CloudNine Infrastructure Ltd.", "address": {"street": "45 Server Farm
Road", "city": "London", "state": null, "country": "United Kingdom",
"postal_code": "EC2V 8RF"}, "tax_id": "GB 312 9847 01", "email":
"contact@cloudnine.co.uk"}, "buyer": {"name": "DataVault Analytics plc",
"address": {"street": "99 Canary Wharf", "city": "London", "state": null,
"country": "United Kingdom", "postal_code": "E14 5AB"}, "tax_id": null,
"email": null}, "line_items": [{"description": "Compute Instances
(c2.xlarge x4, 720 hrs)", "quantity": 720.0, "unit_price": 0.68, "total":
489.60}, {"description": "Storage (20TB SSD, monthly)", "quantity": 1.0,
"unit_price": 840.0, "total": 840.0}, {"description": "CDN Bandwidth (8TB
transfer)", "quantity": 8.0, "unit_price": 32.0, "total": 256.0},
{"description": "Load Balancer (2 units, monthly)", "quantity": 2.0,
"unit_price": 95.0, "total": 190.0}, {"description": "Managed Database
(PostgreSQL, monthly)", "quantity": 1.0, "unit_price": 310.0, "total":
310.0}, {"description": "DDoS Protection (standard tier)", "quantity": 1.0,
"unit_price": 120.0, "total": 120.0}], "subtotal": 2205.60, "tax_amount":
441.12, "total_amount": 2646.72, "currency": "GBP",
"purchase_order_number": null, "notes": "OVERDUE NOTICE: This invoice is
past its due date. Please remit payment immediately. Late payment fee of 2%
per month applies after 30 days."}}
{"sample_id": "004", "file_name": "invoice_004.txt", "expected":
{"invoice_number": "INV-2024-CE-0189", "invoice_date": "2024-03-05",
"due_date": "2024-04-05", "status": "partially_paid", "vendor": {"name":
"CreativeEdge Marketing Pvt. Ltd.", "address": {"street": "501 Maker
Chambers IV, Nariman Point", "city": "Mumbai", "state": null, "country":
"India", "postal_code": "400021"}, "tax_id": "27AAGCC4739R1Z3", "email":
"finance@creativeedge.in"}, "buyer": {"name": "BrandAxis Retail Pvt. Ltd.",
"address": {"street": "14th Floor, DLF Cyber City", "city": "Gurugram",
"state": "Haryana", "country": "India", "postal_code": "122002"}, "tax_id":
null, "email": null}, "line_items": [{"description": "Brand Strategy
Consulting", "quantity": 1.0, "unit_price": 75000.0, "total": 75000.0},
{"description": "Social Media Campaign Design", "quantity": 1.0,
"unit_price": 45000.0, "total": 45000.0}, {"description": "Video Production
(30-sec ad)", "quantity": 2.0, "unit_price": 60000.0, "total": 120000.0}],
"subtotal": 240000.0, "tax_amount": 43200.0, "total_amount": 283200.0,
"currency": "INR", "purchase_order_number": null, "notes": "A partial
payment of ₹1,00,000.00 was received on 15 March 2024. Remaining balance of
₹1,83,200.00 is due by 05 April 2024."}}
{"sample_id": "005", "file_name": "invoice_005.txt", "expected":
{"invoice_number": "INV-2024-PL-0057", "invoice_date": "2024-03-14",
"due_date": null, "status": "unpaid", "vendor": {"name": "PrecisionLab
Instruments Corp.", "address": {"street": "2200 Research Parkway", "city":

```

```

"San Diego", "state": "CA", "country": "United States", "postal_code":
"92121"}, {"tax_id": "47-2381049", "email": "orders@precisionlab.com"},
"buyer": {"name": "GenomeTech Research Institute", "address": {"street":
"500 Science Boulevard", "city": "Boston", "state": "MA", "country":
"United States", "postal_code": "02115"}, "tax_id": null, "email": null},
"line_items": [{"description": "Centrifuge Model CF-5000", "quantity": 2.0,
"unit_price": 4800.0, "total": 9600.0}, {"description": "Micropipette Set
(1000ul)", "quantity": 10.0, "unit_price": 185.0, "total": 1850.0},
{"description": "Lab Refrigeration Unit (4°C)", "quantity": 1.0,
"unit_price": 3200.0, "total": 3200.0}], "subtotal": 14650.0, "tax_amount":
1098.75, "total_amount": 15748.75, "currency": "USD",
"purchase_order_number": null, "notes": "No purchase order on file. Net 45
payment terms apply."}]
{"sample_id": "006", "file_name": "invoice_006.txt", "expected":
{"invoice_number": "INV-2024-GF-0302", "invoice_date": "2024-03-18",
"due_date": "2024-04-17", "status": "paid", "vendor": {"name": "Gulf
Freight Express LLC", "address": {"street": "Warehouse 14, Jebel Ali Free
Zone", "city": "Dubai", "state": null, "country": "UAE", "postal_code":
null}, "tax_id": "100482931400003", "email": "billing@gulffreight.ae"},
"buyer": {"name": "Meridian Trading Company FZCO", "address": {"street":
"Office 2201, DWTC", "city": "Dubai", "state": null, "country": "UAE",
"postal_code": null}, "tax_id": null, "email": null}, "line_items":
[{"description": "Air Freight - DXB to SIN", "quantity": 1.0, "unit_price":
12500.0, "total": 12500.0}, {"description": "Customs Clearance Fees",
"quantity": 1.0, "unit_price": 1800.0, "total": 1800.0}, {"description":
"Warehouse Storage (7 days)", "quantity": 7.0, "unit_price": 350.0,
"total": 2450.0}, {"description": "Documentation Handling", "quantity":
1.0, "unit_price": 600.0, "total": 600.0}], "subtotal": 17350.0,
"tax_amount": 867.50, "total_amount": 18217.50, "currency": "AED",
"purchase_order_number": "PO-MTC-20240318", "notes": "Payment received via
bank transfer on 20 March 2024. Thank you."}]
{"sample_id": "007", "file_name": "invoice_007.txt", "expected":
{"invoice_number": "INV-2024-BF-0061", "invoice_date": "2024-03-22",
"due_date": "2024-04-21", "status": "unpaid", "vendor": {"name": "ByteForge
Hardware Pte. Ltd.", "address": {"street": "18 Tanjong Pagar Road, #05-01",
"city": "Singapore", "state": null, "country": "Singapore", "postal_code":
"088443"}, "tax_id": "M9-0234817-K", "email": "sales@byteforge.sg"},
"buyer": {"name": "Synergy Tech Corp.", "address": {"street": "77 Shenton
Way, Level 12", "city": "Singapore", "state": null, "country": "Singapore",
"postal_code": "078903"}, "tax_id": null, "email": null}, "line_items":
[{"description": "Enterprise Server (Dell R750xa)", "quantity": 2.0,
"unit_price": 8400.0, "total": 16800.0}, {"description": "Network Switch
(48-port)", "quantity": 4.0, "unit_price": 850.0, "total": 3400.0},
{"description": "UPS Battery Backup Unit", "quantity": 2.0, "unit_price":
620.0, "total": 1240.0}], "subtotal": 21440.0, "tax_amount": 1929.60,
"total_amount": 23369.60, "currency": "SGD", "purchase_order_number": null,
"notes": null}]
{"sample_id": "008", "file_name": "invoice_008.txt", "expected":
{"invoice_number": "INV-2024-HC-0088", "invoice_date": "2024-02-01",
"due_date": "2024-03-01", "status": "overdue", "vendor": {"name":
"Harrington & Cole Legal Associates LLP", "address": {"street": "850 Third
Avenue, 22nd Floor", "city": "New York", "state": "NY", "country": "United
States", "postal_code": "10022"}, "tax_id": "13-5527894", "email":
"accounts@harringtoncole.com"}, "buyer": {"name": "Vantage Capital Group

```

```

Inc.", "address": {"street": "200 Park Avenue", "city": "New York",
"state": "NY", "country": "United States", "postal_code": "10166"},
"tax_id": null, "email": null}, "line_items": [{"description": "M&A Due
Diligence Review", "quantity": 40.0, "unit_price": 450.0, "total":
18000.0}, {"description": "Contract Negotiation Support", "quantity": 15.0,
"unit_price": 450.0, "total": 6750.0}, {"description": "Regulatory
Compliance Advisory", "quantity": 10.0, "unit_price": 450.0, "total":
4500.0}], "subtotal": 29250.0, "tax_amount": 0.0, "total_amount": 29250.0,
"currency": "USD", "purchase_order_number": null, "notes": "OVERDUE NOTICE:
This invoice was due on 01 March 2024. A late payment penalty of 1.5% per
month is now accruing."}}
{"sample_id": "009", "file_name": "invoice_009.txt", "expected":
{"invoice_number": "INV-2024-PM-0214", "invoice_date": "2024-04-08",
"due_date": "2024-05-08", "status": "unpaid", "vendor": {"name": "Pinnacle
Media Group Inc.", "address": {"street": "340 King Street West, Suite 500",
"city": "Toronto", "state": "ON", "country": "Canada", "postal_code": "M5V
1J2"}, "tax_id": "82471-9203-RT0001", "email": "billing@pinnaclemedia.ca"},
"buyer": {"name": "Northern Lights Retail Corp.", "address": {"street":
"1000 Portage Avenue", "city": "Winnipeg", "state": "MB", "country":
"Canada", "postal_code": "R3G 0S8"}, "tax_id": null, "email": null},
"line_items": [{"description": "Advertising Retainer (April 2024)",
"quantity": 1.0, "unit_price": 12000.0, "total": 12000.0}, {"description":
"Performance Analytics Report", "quantity": 1.0, "unit_price": 2500.0,
"total": 2500.0}, {"description": "Creative Asset Production", "quantity":
1.0, "unit_price": 4200.0, "total": 4200.0}], "subtotal": 18700.0,
"tax_amount": 2431.0, "total_amount": 21131.0, "currency": "CAD",
"purchase_order_number": "PO-CA-7721", "notes": "Payment terms: Net 30. E-
transfer or wire accepted."}}
{"sample_id": "010", "file_name": "invoice_010.txt", "expected":
{"invoice_number": "INV-2024-CB-0503", "invoice_date": "2024-04-15",
"due_date": "2024-05-15", "status": "partially_paid", "vendor": {"name":
"Cape Build Materials (Pty) Ltd.", "address": {"street": "12 Industria
Road", "city": "Cape Town", "state": null, "country": "South Africa",
"postal_code": "7490"}, "tax_id": "4320198754", "email":
"accounts@capebuild.co.za"}, "buyer": {"name": "Sunrise Property Developers
(Pty) Ltd.", "address": {"street": "88 Bree Street", "city":
"Johannesburg", "state": null, "country": "South Africa", "postal_code":
"2001"}, "tax_id": null, "email": null}, "line_items": [{"description":
"Concrete Blocks (Standard)", "quantity": 5000.0, "unit_price": 8.50,
"total": 42500.0}, {"description": "River Sand (per ton)", "quantity":
40.0, "unit_price": 480.0, "total": 19200.0}, {"description": "Building
Cement (50kg bags)", "quantity": 200.0, "unit_price": 145.0, "total":
29000.0}, {"description": "Steel Reinforcement Bars (12mm)", "quantity":
300.0, "unit_price": 85.0, "total": 25500.0}, {"description":
"Waterproofing Membrane (rolls)", "quantity": 50.0, "unit_price": 320.0,
"total": 16000.0}, {"description": "Roof Tiles (per sqm)", "quantity":
120.0, "unit_price": 265.0, "total": 31800.0}, {"description": "PVC Piping
(4m lengths, 50mm)", "quantity": 100.0, "unit_price": 75.0, "total":
7500.0}, {"description": "Delivery Charges", "quantity": 1.0, "unit_price":
4500.0, "total": 4500.0}], "subtotal": 176000.0, "tax_amount": 26400.0,
"total_amount": 202400.0, "currency": "ZAR", "purchase_order_number": null,
"notes": "A partial payment of ZAR 100,000.00 was received on 20 April
2024. Outstanding balance: ZAR 102,400.00. Please remit by 15 May 2024."}}

```

Important: Each line above is a single JSON object. When you paste this into `ground_truth.jsonl`, make sure there are no blank lines between records and no trailing commas.

7. DeepSeek Prompts

Use these if you want to regenerate any file from scratch in DeepSeek Chat. Each prompt is self-contained.

Prompt: Generate `schema.py`

Create a Python file called `schema.py` for a B2B invoice extraction project using Pydantic v2.

Requirements:

- Python 3.10+
- Use Pydantic BaseModel and Field
- Define an InvoiceStatus enum with values: paid, unpaid, overdue, partially_paid, unknown
- Define an Address model with fields: street (str), city (str), state (Optional[str]), country (str), postal_code (Optional[str])
- Define a Party model with fields: name (str), address (Address), tax_id (Optional[str]), email (Optional[str])
- Define a LineItem model with fields: description (str), quantity (float), unit_price (float), total (float)
- Define an Invoice model with fields:
 - invoice_number: str
 - invoice_date: str (with Field description: "ISO date string YYYY-MM-DD")
 - due_date: Optional[str] (with Field description: "ISO date string YYYY-MM-DD")
 - status: InvoiceStatus
 - vendor: Party
 - buyer: Party
 - line_items: List[LineItem]
 - subtotal: float
 - tax_amount: float
 - total_amount: float
 - currency: str (with Field description: "Three-letter ISO currency code")
 - purchase_order_number: Optional[str]
 - notes: Optional[str]

Return only the complete Python file content. No explanation.

Prompt: Generate `main.py`

Create a Python file called `main.py` for a B2B invoice extraction pipeline using the Google Gemini API.

Requirements:

- Python 3.10+
- CLI usage: `python main.py samples/invoice_001.txt`
- Read the `GEMINI_API_KEY` from environment variable; if missing, print setup instructions and exit with code 1
- If the input file does not exist, print a clear error and exit with code 1
- If the file is empty, print a clear error and exit with code 1
- Send the invoice text to Gemini (model: `gemini-1.5-flash`) with `response_mime_type="application/json"` and a `response_schema` derived from the Invoice Pydantic model
- Parse the returned JSON; if malformed, print the raw response and exit with code 1
- Validate the parsed JSON using Pydantic's `Invoice.model_validate()`; if validation fails, print the `ValidationError` and exit with code 1
- Print the extracted invoice as pretty JSON to stdout
- Save the output to `outputs/{stem}.extracted.json`
- Use the `google-generativeai` SDK (`import google.generativeai as genai`)
- Import `Invoice` from `schema.py`

Include a helper function `pydantic_to_gemini_schema()` that converts a Pydantic model's JSON schema to the nested dict format Gemini's `response_schema` expects (uppercase TYPE strings like `"OBJECT"`, `"STRING"`, `"NUMBER"`, `"ARRAY"`, resolving `$ref` references from `$defs`).

Include a detailed extraction prompt string that instructs Gemini to extract all invoice fields, return dates in `YYYY-MM-DD` format, use three-letter currency codes, and use null for absent optional fields.

Return only the complete Python file content. No explanation.

Prompt: Generate `evaluate.py`

Create a Python file called `evaluate.py` that evaluates invoice extraction accuracy.

Requirements:

- Python 3.10+
- No external dependencies beyond the standard library
- Load `ground_truth.jsonl` (one JSON object per line, fields: `sample_id`, `file_name`, `expected`)
- For each record, read `outputs/{stem}.extracted.json`
- Flatten both `extracted` and `ground_truth` dicts using dot notation (e.g. `vendor.address.city`)
- For list fields (like `line_items`), compare by converting to canonical JSON strings

- Normalize values before comparison: strip whitespace, lowercase strings, round floats to 2 decimal places
- Write field_accuracy.csv with columns: sample_id, field_name, extracted_value, ground_truth_value, exact_match
- Print overall accuracy (correct / total fields) and a per-field accuracy table to stdout
- If extracted files are missing, print a warning listing them and continue with the rest
- If ground_truth.jsonl is missing, print an error and exit

Return only the complete Python file content. No explanation.

Prompt: Generate `generate_samples.py`

Create a Python file called `generate_samples.py` that uses the Gemini API to generate 10 synthetic B2B invoice text files.

Requirements:

- Python 3.10+
- Use google-generativeai SDK (model: gemini-1.5-flash)
- Read GEMINI_API_KEY from environment; exit with code 1 if missing
- Save files to `samples/invoice_001.txt` through `invoice_010.txt`
- Skip files that already exist
- Each invoice should be plain text (not JSON or markdown)
- The 10 invoices should cover: USD software consulting (with PO), EUR office furniture (paid), GBP cloud hosting multi-line-item (overdue), INR marketing (partially paid), USD lab equipment (no due date), AED freight (paid, with PO), SGD IT hardware (unusual spacing), USD legal (overdue), CAD advertising retainer (with PO), ZAR construction materials (partially paid, 8 line items)
- Use a clear prompt template that asks Gemini for realistic company names, addresses, line items, and payment notes matching each scenario

Return only the complete Python file content. No explanation.

Prompt: Generate 10 Invoice Samples

Generate 10 synthetic B2B invoice plain-text files. Return each one clearly labeled with its filename.

Invoice 001: USD software consulting, vendor TechPulse Solutions, buyer Nexora Digital, status UNPAID, PO-2024-0091, 3 line items, 8% tax.

Invoice 002: EUR office furniture, German vendor, Belgian buyer, status PAID with payment note, 3 line items, 19% VAT.

Invoice 003: GBP cloud hosting, 6 line items including compute, storage, CDN, load balancer, database, DDoS protection, status OVERDUE with late fee note.

Invoice 004: INR marketing services, Indian vendor with GSTIN, Indian buyer, status PARTIALLY PAID with partial payment note, 18% GST.
 Invoice 005: USD lab equipment, status UNPAID, NO due date present at all, 3 items, 7.5% tax.
 Invoice 006: AED freight and logistics, Dubai vendor with TRN, Dubai buyer, status PAID with payment confirmation, 5% VAT, include PO.
 Invoice 007: SGD IT hardware, Singapore vendor, status UNPAID, use irregular spacing and alignment to simulate a poorly formatted scan, 3 items, 9% GST.
 Invoice 008: USD legal consulting, New York law firm, status OVERDUE with late penalty notice, 0% tax (legal services), 3 items, no PO.
 Invoice 009: CAD advertising retainer, Toronto vendor, Winnipeg buyer, status UNPAID, include PO-CA-7721, 13% HST.
 Invoice 010: ZAR construction materials, Cape Town vendor, Johannesburg buyer, status PARTIALLY PAID with outstanding balance note, 8 line items, 15% VAT.

Each invoice must have: vendor name, address, tax ID, email; buyer name, address; invoice number, date, due date (except 005); currency; all line items with qty, unit price, total; subtotal, tax, total.

Return only the plain text of each invoice separated by its filename.

Prompt: Generate `ground_truth.jsonl`

Create a `ground_truth.jsonl` file with 10 lines, one JSON object per line.

Each object must have exactly these top-level keys: `sample_id` (string "001" to "010"), `file_name` (e.g. "invoice_001.txt"), `expected` (a full Invoice object matching this schema):

```
{
  invoice_number: str,
  invoice_date: str (YYYY-MM-DD),
  due_date: str or null,
  status: "paid" | "unpaid" | "overdue" | "partially_paid" | "unknown",
  vendor: { name, address: { street, city, state, country, postal_code },
    tax_id, email },
  buyer: { name, address: { street, city, state, country, postal_code },
    tax_id, email },
  line_items: [{ description, quantity, unit_price, total }],
  subtotal: float,
  tax_amount: float,
  total_amount: float,
  currency: str (3-letter ISO),
  purchase_order_number: str or null,
  notes: str or null
}
```

Use the following invoice specifications to fill in the expected values:

[Paste your 10 invoice samples here and ask DeepSeek to extract the ground truth JSON for each.]

Return only the 10 JSON lines with no blank lines between them.

Prompt: Generate `README.md`

Create a README.md for a Python project called "B2B Invoice Extractor" that uses the Google Gemini API and Pydantic to extract structured data from plain-text invoices.

Include these sections:

1. What the project does (2-3 sentences)
2. Project structure (file tree)
3. Prerequisites (Python 3.10+, Gemini API key)
4. Installation (pip install -r requirements.txt)
5. Setup (export GEMINI_API_KEY=...)
6. Generating samples (python generate_samples.py)
7. Running extraction on one file (python main.py samples/invoice_001.txt)
8. Running extraction on all samples (bash loop or Python loop)
9. Running evaluation (python evaluate.py)
10. Expected outputs (field_accuracy.csv, accuracy_summary.md, outputs/ directory)
11. Error handling notes (what errors look like and what to check)

Write in clear markdown. Keep it practical and concise.

Prompt: Generate `accuracy_summary.md`

Create an accuracy_summary.md template for a document extraction evaluation report.

The report should cover results from running a Gemini-based extractor on 10 B2B invoice samples.

Include these sections:

1. Overview (date, number of samples, overall accuracy %)
2. Per-field accuracy table (field name, correct count, total count, accuracy %)
3. Fields that performed well (with brief explanation why)
4. Fields that failed most (with brief explanation why)
5. Common error types (enum mismatch, date format, numeric rounding, optional field handling, list item mismatch)
6. Likely root causes for each error type
7. Ideas to improve accuracy (prompt engineering, schema tweaks, preprocessing)
8. Next steps

Fill the table and sections with placeholder values like [X]% that the intern should replace after running `evaluate.py`.

8. Evaluation Logic

`evaluate.py` (already given in Section 4) works as follows:

Step 1 — Load ground truth. Read each line of `ground_truth.jsonl` and parse the JSON.

Step 2 — Load extracted output. For each ground truth record, find the matching file in `outputs/`. If it's missing, warn and skip.

Step 3 — Flatten both dicts. The `flatten()` function converts nested structures into dot-notation keys:

```
vendor.name          → "TechPulse Solutions Inc."
vendor.address.city  → "Austin"
line_items           → '["description": "...", ...]' ← stored as JSON
string
```

List fields like `line_items` are compared as their full canonical JSON representation. This is a strict comparison — even order matters. If Gemini reorders line items, it fails. This is intentional: it forces you to think about whether that matters in production.

Step 4 — Compare. Values are normalized (stripped, lowercased, floats rounded to 2dp) before comparison. Boolean `exact_match` is written per field.

Step 5 — Write CSV. `field_accuracy.csv` has one row per (sample, field) pair.

Step 6 — Print summary. Overall accuracy and per-field accuracy table printed to stdout.

Running the full evaluation pipeline

```
# Step 1: Extract all 10 invoices
for i in 001 002 003 004 005 006 007 008 009 010; do
    python main.py samples/invoice_${i}.txt
done

# Step 2: Evaluate
python evaluate.py
```

9. Accuracy Summary Template

Save this as `accuracy_summary.md` and fill it in after running `evaluate.py`.

Invoice Extraction Accuracy Summary

****Date:**** YYYY-MM-DD
****Model:**** gemini-1.5-flash
****Samples evaluated:**** 10
****Overall field accuracy:**** [X]%

Per-Field Accuracy

Field	Correct	Total	Accuracy
invoice_number	[n]	10	[x]%
invoice_date	[n]	10	[x]%
due_date	[n]	10	[x]%
status	[n]	10	[x]%
vendor.name	[n]	10	[x]%
vendor.address.street	[n]	10	[x]%
vendor.address.city	[n]	10	[x]%
vendor.address.country	[n]	10	[x]%
vendor.tax_id	[n]	10	[x]%
vendor.email	[n]	10	[x]%
buyer.name	[n]	10	[x]%
buyer.address.city	[n]	10	[x]%
line_items	[n]	10	[x]%
subtotal	[n]	10	[x]%
tax_amount	[n]	10	[x]%
total_amount	[n]	10	[x]%
currency	[n]	10	[x]%
purchase_order_number	[n]	10	[x]%
notes	[n]	10	[x]%

Fields That Performed Well

- ****invoice_number, currency:**** Highly structured; Gemini extracts these reliably.
- ****vendor/buyer names:**** Usually clearly labeled on invoices.

Fields That Failed Most

- ****line_items:**** _[Describe what happened – reordering, missing items, rounding errors]_
- ****status:**** _[Did the model guess based on wording? Did it default to "unknown"?]_
- ****due_date:**** _[Did the model hallucinate a date when none was present?]_
- ****notes:**** _[Was it too long, truncated, or rephrased?]_

Common Error Types

Error Type	Description
Enum mismatch	Model returned "Partially Paid" instead of "partially_paid"
Date format	Returned "15 March 2024" instead of "2024-03-15"
Numeric rounding	tax_amount off by ± 0.01 due to floating point
Optional field	Returned empty string instead of null
List mismatch	Line items reordered or description slightly different

Likely Root Causes

- Enum mismatch: response_schema enum values not enforced at the string level
- Date format: prompt instruction not strong enough; add "Return ONLY in YYYY-MM-DD format. Never use any other date format."
- Numeric: floating point representation; consider rounding in schema or evaluation
- Optional fields: schema didn't differentiate null vs absent; Pydantic default=None helped but model may still return ""

Ideas to Improve Accuracy

1. Strengthen prompt: add explicit examples of acceptable date/enum formats
2. Add a pre-processing step that normalizes dates in the extracted JSON before Pydantic validation
3. For line_items, sort by description before comparison in evaluate.py
4. Add a retry loop: if validation fails, send Gemini the error message and ask it to correct the JSON
5. Consider extracting to a less strict schema first, then mapping to the strict one

Next Steps

- [] Fix the top 3 failing fields and re-run
- [] Try gemini-1.5-pro and compare accuracy
- [] Add the correction retry loop to main.py
- [] Write learnings.md

10. README

B2B Invoice Extractor

Extracts structured data from plain-text B2B invoices using the Google Gemini API and validates the output with Pydantic.

This is a Day 3 intern project at Wohlig, simulating real KYC and invoice processing workflows.

Project Structure

extractor/ main.py # Core extraction pipeline schema.py # Pydantic models and enums evaluate.py # Accuracy evaluation against ground truth generate_samples.py # Generate synthetic invoice samples using Gemini requirements.txt ground_truth.jsonl # Expected outputs for all 10 samples samples/ # Plain-text invoice files outputs/ # Extracted JSON files (created by main.py) field_accuracy.csv # Created by evaluate.py accuracy_summary.md # Fill in manually after evaluation learnings.md # Your reflection notes

Prerequisites

- Python 3.10 or higher
- A Gemini API key (free at <https://aistudio.google.com/app/apikey>)

Installation

```
```bash
cd extractor
pip install -r requirements.txt
```

## Setup

```
export GEMINI_API_KEY="your-api-key-here"
```

On Windows (Command Prompt):

```
set GEMINI_API_KEY=your-api-key-here
```

## Generating Samples

If you want Gemini to generate the invoice samples (instead of using the hand-written ones from the guide):

```
python generate_samples.py
```

This creates `samples/invoice_001.txt` through `samples/invoice_010.txt`.

## Running Extraction

Single file:

```
python main.py samples/invoice_001.txt
```

All files:

```
for i in 001 002 003 004 005 006 007 008 009 010; do
 python main.py samples/invoice_${i}.txt
done
```

Extracted JSON files are saved to `outputs/`.

## Running Evaluation

```
python evaluate.py
```

This reads `ground_truth.jsonl` and all extracted JSONs, writes `field_accuracy.csv`, and prints accuracy to the console.

## Expected Outputs

After running extraction on all 10 samples:

- `outputs/invoice_001.extracted.json` through `invoice_010.extracted.json`

After running evaluation:

- `field_accuracy.csv` — row per (sample, field) with `exact_match` column
- Fill in `accuracy_summary.md` with the printed numbers

## Error Handling

Error	What to check
<code>GEMINI_API_KEY</code> not set	Run <code>export GEMINI_API_KEY=...</code>
<code>File not found</code>	Check the path you passed to <code>main.py</code>

Error	What to check
Malformed JSON	Gemini returned something unexpected; check the raw output printed
Pydantic validation failed	The JSON structure doesn't match the schema; check the error details
Empty response	Rate limit hit or model returned nothing; wait and retry

```

11. `learnings.md` Template

```markdown
# Extraction Learnings

Fill this in as you work. Be specific. Vague notes are useless.

---

## What broke?

_Describe the issue exactly. Which file? Which field? What did you see?_

---

## Was it a model issue, schema issue, document quality issue, or
evaluation issue?

- **Model issue:** Gemini misread the document or hallucinated a value
- **Schema issue:** The Pydantic model or response_schema was wrong
- **Document quality issue:** The sample invoice was ambiguous or poorly
formatted
- **Evaluation issue:** Your comparison logic was wrong (e.g., comparing
floats without rounding)

---

## What did I expect?

_What value did you expect the model to return?_

---

## What did I get?

_Paste the actual extracted value here._

---

## How did I debug it?
```

```

_What steps did you take? (e.g., printed the raw Gemini response, checked
the schema, looked at the evaluate.py comparison logic)_

---

## What change fixed it?

_Describe the fix exactly. Which file did you change? What did you change?_

---

## Field-Specific Notes

| Field | Failed on Sample | Root Cause | Fix Applied |
|---|---|---|---|
| status | 004, 010 | Model returned "Partially Paid" | Added explicit enum
list to prompt |
| due_date | 005 | Model hallucinated a date | Added "return null if not
present" to prompt |
| line_items | 003 | Reordered by model | Sorted by description in
evaluate.py |

---

## What would I do differently in production?

1. _..._
2. _..._
3. _..._

---

## Questions I still have

_List anything you're still unsure about._

```

12. Troubleshooting

Missing API key

Symptom: `[ERROR] GEMINI_API_KEY environment variable is not set.`

Fix:

```
export GEMINI_API_KEY="your-key-here"
```

Get a key at <https://aistudio.google.com/app/apikey>. The free tier is sufficient for this project.

Invalid Gemini SDK import

Symptom: `ModuleNotFoundError: No module named 'google.generativeai'`

Fix:

```
pip install google-generativeai
```

If you installed with pip and it still fails, make sure you're using the same Python environment:

```
which python
which pip
python -m pip install google-generativeai
```

Pydantic validation errors

Symptom: `[ERROR] Pydantic validation failed.` followed by a list of field errors.

What it means: Gemini returned JSON that doesn't match the schema. Common reasons:

- `status` field had a value not in the enum (e.g. `"Partially Paid"` instead of `"partially_paid"`)
- A required field like `subtotal` came back as a string instead of a float
- A nested object like `vendor.address` was missing a required field

Fix: Print the raw JSON (already done by `main.py` on error). Look at the validation error message — it tells you the exact field and expected type. Then either fix the prompt to be more explicit, or check if your `pydantic_to_gemini_schema()` function is correctly mapping the field.

JSON schema mismatch

Symptom: Gemini returns JSON but it has a different structure than expected.

Example: `vendor` comes back as a flat string instead of a nested object.

Fix: Check your `pydantic_to_gemini_schema()` output by printing it before the API call:

```
import json
print(json.dumps(pydantic_to_gemini_schema(Invoice), indent=2))
```

Make sure nested objects have `"type": "OBJECT"` and `"properties"` keys. If `$ref` references aren't being resolved, the schema will be incomplete.

Empty model response

Symptom: [ERROR] Gemini returned an empty response.

Causes:

- Rate limit hit (free tier: 15 requests/minute on gemini-1.5-flash)
- Safety filter triggered (unlikely for invoices)
- Very large document that exceeded context

Fix: Wait 60 seconds and retry. For rate limits, add a sleep:

```
import time
time.sleep(4) # 4 seconds between calls = ~15/min
```

File path issues

Symptom: [ERROR] File not found: samples/invoice_001.txt

Fix: Make sure you're running `main.py` from inside the `extractor/` directory:

```
cd extractor
python main.py samples/invoice_001.txt
```

Not from a parent directory. The path is relative to where you run the script.

Rate limits

Symptom: `google.api_core.exceptions.ResourceExhausted: 429`

Fix: Add delays between calls. When running all 10 samples in a loop:

```
for i in 001 002 003 004 005 006 007 008 009 010; do
  python main.py samples/invoice_${i}.txt
  sleep 5
done
```

Bad extracted totals

Symptom: `subtotal` or `total_amount` is wrong, or `line_item.total` doesn't match `quantity × unit_price`.

What usually happens: Gemini reads the formatted number correctly but loses precision — e.g. `1,233.10` becomes `1233.1` which rounds to `1233.1` but the ground truth has `1233.10`. The evaluation normalizes floats to 2dp, so this should pass. If it doesn't, check whether Gemini returned a string instead of a float.

Harder case: The invoice has a currency symbol embedded (e.g. ₹2,40,000.00) and Gemini returns 240000 without the comma-parsing. That's correct behavior — Gemini should strip formatting.

Fix: Add to prompt: "Return all numeric amounts as plain numbers without currency symbols, commas, or formatting."

Optional fields missing

Symptom: `purchase_order_number` is `None` in the output even though it's in the invoice (or vice versa).

Fix:

- If the field IS in the invoice but extraction returned `null`: the prompt may not mention it explicitly. Add it to the field list in the prompt.
 - If the field is NOT in the invoice but extraction returned a value: Gemini hallucinated it. Add to prompt: "If a field is not clearly stated in the document, return null. Do not infer or guess optional fields."
-

Enum mismatch

Symptom: `status` extraction fails validation with `Input should be 'paid', 'unpaid', 'overdue', 'partially_paid' or 'unknown'.`

What happened: Gemini returned something like `"Partially Paid"` or `"OVERDUE"` instead of the lowercase snake_case enum value.

Fix: Make the enum values explicit in the prompt:

```
status must be exactly one of these strings (case-sensitive):  
paid, unpaid, overdue, partially_paid, unknown
```

```
Read the invoice for clues: "Payment received" → paid, "OVERDUE NOTICE" →  
overdue,  
"partial payment was received" → partially_paid, "UNPAID" → unpaid.
```

The `response_schema` with enum values should also constrain this, but the constraint is not always 100% enforced for string casing. Be explicit in the prompt.

End of guide.